

PDC



Assessments # 01

Submitted By

QASIM ALI (20P-0070)

Submitted to:

Section: BSCS-6A

**DEPARTMENT OF COMPUTER SCIENCE
FAST NATIONAL UNIVERSITY OF COMPUTER
AND EMERGING SCIENCES, PESHAWAR**

Task 1:

Ans:

1. Provide a list of run-time routines that are used in OpenMP?
2. Why aren't you seeing the Hello World output thread sequence as 0, 1, 2, 3 etc. Why are they disordered?
3. What happens to the thread_id if you change its scope to before the pragma?
4. Convert the code to serial code

1. Provide a list of run-time routines that are used in OpenMP?

ANS:

omp_get_thread_num(): Returns the thread number of the calling thread.

omp_get_num_threads(): Returns the total number of threads in the current team.

omp_set_num_threads(): Sets the number of threads to be used for subsequent parallel regions.

omp_get_max_threads(): Returns the maximum number of threads that could be used.

omp_get_num_procs(): Returns the number of processors available.

2: The order of output from the parallel region is not guaranteed to be in sequence (0, 1, 2, 3, etc.) because the order in which threads execute is not specified in OpenMP. The scheduling of threads is managed by the operating system and the OpenMP runtime. Each thread executes independently, and their order of execution is not deterministic.

3: If we change the scope of thread_id to before the pragma, it becomes a shared variable, and each thread will have its own copy. This may lead to unexpected behavior because each thread will be modifying its own copy of thread_id. It's generally safer to declare thread_id inside the parallel region to ensure each thread has its own private copy.

4: serial code?

```
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main() {
5      #pragma omp parallel{
6          int thread_id = omp_get_thread_num();
7          printf("Hello, world! This is thread %d\n", thread_id);
8      }
9      return 0;
10 }
```

Task 2:

2. Convert it into Parallel, such that only the addition part is parallelized

```
1  #include <stdio.h>
2  int main() {
3      int array1[16] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, ..., 16};
4      int array2[16] = {16, ..., 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1};
5      int result[16];
6      for (int i = 0; i < 16; i++)
7          { result[i] = array1[i] + array2[i];
8          }
9      for (int i = 0; i < 16; i++)
10         { printf("%d ", result[i]);
11         }
12     printf("\n");
13     return 0;
14 }
```

3. The display loop at the end displays the result. Modify the code such that this is also parallel, but only thread of id 0 is able to display the entire loop. The others should not do anything. When making it parallel, make sure its the old threads and new threads are not created. What output do you see?

```
1  #include <stdio.h>
2  int main() {
3      int array1[16] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, ..., 16};
4      int array2[16] = {16, ..., 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1};
5      int result1 = 0, result2 = 0;
6      for (int i = 0; i < 16; i++)
7          { result1 += array1[i];
8          }
9      if (result1 > 10)
10         { result2 = result1;
11           for (int i = 0; i < 16; i++)
12               { result2 += array2[i];
13               }
14         }
15     printf("%d\n", result2);
16     return 0;
17 }
```

Output:

```
(root@kali)-[~/openMP]
# gcc -o t3 -fopenmp task2-b.c

(root@kali)-[~/openMP]
# ./t3
17 17 17 17 17 17 17 17 17 17 17 17 17 17 17 17
```

Task 3

1. Modify the code of Task 2 and do the job in half of the threads.

```
1  include <stdio.h>
2  #include <omp.h>
3
4  int main() {
5      int array1[16] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
6      int array2 [16] {16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1};
7      int result[16];
8
9      //Get the total number of thread
10     int total_threads = omp_get_max_threads();
11
12     //use only half of the thread
13     int omp_get_thread_to_num = total_threads/2;
14
15     #pragma omp parallel for num_threads(omp_get_thread_to_use)
16     for (int i=0; i < 16; i++) {
17         result[i] = array1[i] + array2[i];
18     }
19
20     //dispaly the result
21     for(int i=0; i<16; i++){
22         printf("%d", result[i]);
23     }
24     printf("\n");
25     return 0;
26 }
27 |
```

Task 4 1. The following code adds the contents of the array array1 and array2.

```
#include <stdio.h>
```

```
int main() {
```

```
int array1[16] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, ..., 16};
```

```
int array2[16] = {16, ..., 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1};
```

```
int result1 = 0, result2 = 0;
```

```
for (int i = 0; i < 16; i++){
```

```
    result1 += array1[i];
```

```
}
```

```
if (result1 > 10) {
```

```
    result2 = result1;
```

```

    for (int i = 0; i < 16; i++) {
        result2 += array2[i];
    }
}

printf("%d\n", result2);

return 0;
}

```

2. Convert it into Parallel using 16 threads.

```

1  #include <stdio.h>
2  int main() {
3      int array1[16] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, ..., 16};
4      int array2[16] = {16, ..., 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1};
5      int result1 = 0, result2 = 0;
6
7
8      #pragma omp parallel num_threads(16)
9      {
10         #pragma omp for reduction(+:result1)
11         for (int i = 0; i < 16; i++){
12             result1 += array1[i];
13         }
14         #pragma omp section
15         {
16             #pragma omp section
17             {
18                 if (result1 > 10){
19                     #pragma omp parallel for reduction(+:result2)
20                     for(int i=0; i< 16; i++){
21                         result2 +=array2[i];
22                     }
23                 }
24             }
25         }
26
27         printf("%d\n", result2);
28
29         return 0;
30     }

```

3. Try removing the reduction() clause and add #pragma omp atomic just before the +=. What is the effect on result? Explain?

```

1  #include <stdio.h>
2  int main() {
3      int array1[16] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, ..., 16};
4      int array2[16] = {16, ..., 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1};
5      int result1 = 0, result2 = 0;
6
7
8      #pragma omp parallel num_threads(16)
9      {
10         #pragma omp for
11         for (int i = 0; i < 16; i++){
12             #pragma omp atomic
13             result1 += array1[i];
14         }
15         #pragma omp section
16         {
17             #pragma omp section
18             {
19                 if (result1 > 10){
20                     #pragma omp parallel for
21                     for(int i=0; i< 16; i++){
22                         #pragma omp atomic
23                         result2 +=array2[i];
24                     }
25                 }
26             }
27         }
28
29         printf("%d\n", result2);
30
31         return 0;
32     }

```

Effect of result: The `#pragma omp atomic` directive ensures that the specified operation is executed atomically, avoiding race conditions that may occur in parallel regions. However, using atomic operations can introduce contention, and in some cases, it might lead to decreased performance compared to using a reduction clause. In this specific code, since the updates to `result1` and `result2` are performed atomically, the final result should still be correct. However, the performance characteristics may vary depending on the specifics of the system and workload.

Task 5:

```
File Actions Edit View Help
(kali@kali)-[~]
└─$ ./gprof_test
Sum in funcA: 5050
Product in funcB: 3628800
Memory allocated in funcC

(kali@kali)-[~]
└─$ gprof gprof_test gmon.out
Flat profile:

Each sample counts as 0.01 seconds.
no time accumulated

% cumulative self self total
time seconds seconds calls Ts/call Ts/call name
0.00 0.00 0.00 1 0.00 0.00 funcA
0.00 0.00 0.00 1 0.00 0.00 funcB
0.00 0.00 0.00 1 0.00 0.00 funcC

% the percentage of the total running time of the
time program used by this function.

cumulative a running sum of the number of seconds accounted
seconds for by this function and those listed above it.

self the number of seconds accounted for by this
seconds function alone. This is the major sort for this
listing.

calls the number of times this function was invoked, if
this function is profiled, else blank.

self the average number of milliseconds spent in this
ms/call function per call, if this function is profiled,
```

Call graph (explanation follows)

granularity: each sample hit covers 2 byte(s) no time propagated

index	% time	self	children	called	name
		0.00	0.00	1/1	main [9]
[1]	0.0	0.00	0.00	1	funcA [1]
		0.00	0.00	1/1	main [9]
[2]	0.0	0.00	0.00	1	funcB [2]
		0.00	0.00	1/1	main [9]
[3]	0.0	0.00	0.00	1	funcC [3]

This table describes the call tree of the program, and was sorted by the total amount of time spent in each function and its children.

Index by function name

[1] funcA

[2] funcB

[3] funcC

```
(kali@kali)-[~]
└─$ gprof gprof_test gmon.out | gprof2dot | dot -Tpng -o output.png
```

funcA
100.00%
(100.00%)
1x

funcB
100.00%
(100.00%)
1x

funcC
100.00%
(100.00%)
1x